



CS 470 Project One Guidelines and Rubric

Competency

In this project, you will demonstrate your mastery of the following competencies:

- Apply cloud-based development principles and best practices in application development
- Develop applications that run on cloud-based frameworks

Scenario

You are a developer for **QnA**, a startup company. You just finished developing a full stack web application. After implementing in-house servers, you watched app usage take off. It's an exciting time for the company. The company is growing, and you can feel the growing pains. The company is expecting a drastic increase in customers. The once-fluid office environment allowed customers to choose their favorite devices and platforms. But you've noticed that this environment has led to major limitations in collaboration. Development has become siloed. Sometimes, only one key person can manage certain aspects of the application. You need a solution for moving the application to a cloud-based framework. Moving to the cloud will allow your application to expand with the growing company.

In this project, you will explore cloud development frameworks for using cloud infrastructure to support your application. This project will allow you to migrate your web application from the desktop to a cloud-based environment. These iterations will show you how the cloud is designed to make scaling and growing your business more efficient. The IT team has determined that Amazon Web Services (AWS) provides a **Platform as a Service** capability that allows you to minimize system management and efficiently run services by dynamically scaling an application's capacity to run up and down. You will be leveraging the principle of elasticity.

Directions

Specifically, your web application must demonstrate your ability to do the following actions:

Frontend: Establish an S3 server to host a static Angular website.

- Prepare the client-side and web server setup, and apply the principles of serverless web hosting by moving frontend-Angular to the cloud.
- Prepare S3 storage buckets and objects and IAM security settings.

Now that you have created your containers, you can see this approach's benefits and pitfalls. Your director wants to make certain that your site is elastic and secure, can process data quickly, and can keep up with the traffic and transactions as the number of customers increases. She wants to use more AWS products and move the frontend to S3, a serverless solution.

You will run your application in a serverless environment using a cloud-based database solution. You will detach the Angular frontend and run it in the cloud, removing the need for the original server.

Backend and API: Migrate all of the server-side business logic.

- Demonstrate Lambda compute models using Lambda containers to distribute functionality in a serverless environment.
- Migrate all of the server-side business logic, apply principles of serverless API, and demonstrate how they give the correct results when Lambda executes.
- Correctly create a database with the appropriate structure that is able to perform CRUD functions.
- Manually enable CORS as defined on the CORS on AWS API Gateway website.
- Deploy the completed API using the “api” deployment name.
- Apply changes to the LoopBack SDK configuration in the client, modify environment projects, and build and deploy the Angular app. The Angular app should run on AWS, calling through Lambda to DynamoDB, and successfully allow CRUD functions on Question and Answer.
- Set the permissions through policies and configure them to enable all of the other services with access to read-write and read-only.

As you explore native cloud services, you will discover best practices that simplify a developer's work. This lets the developer focus on developing rather than continuously working on maintaining the existing infrastructure.

In the final phase, you will deconstruct the full stack application APIs and turn them into microservices, making them fully cloud-native applications. In this phase, you will add CRUD functionality. You will also create an API Gateway using Lambdas, eliminating the need for the original MEAN server.

What to Submit

To complete this project, you must submit the following to Brightspace:

- API URL / Endpoint: Backend of the project running.
- S3 URL / Endpoint: Frontend of the project hosted on S3.
- Screenshots for Each Part/Section:
 - Correspond to different parts/sections in the Project One Guide (Evidence of S3 setup, IAM security, Lambda functions, CORS configurations, etc.)
 - Screenshots must show the complete browser window with the student logged into AWS Academy.

Supporting Materials

The following resource may help support your work on the project:

Resource: [Project One Guide](#)

This guide supplies specific instructions for this project.

Project One Rubric

Criteria	Exceeds Expectations	Meets Expectations	Partially Meets Expectations	Does Not Meet Expectations	Value
Serverless Web Hosting	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Prepares the client-side web server setup and applies the principles of serverless web hosting by moving frontend-Angular to the cloud (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include the omission of a URL, a screenshot of the server files, or the dist directory; website must function fully and support CRUD capabilities (55%)	Does not attempt criterion (0%)	5
Lambda Compute Models	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Demonstrates Lambda compute models by using Lambda containers to distribute functionality in a serverless environment (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include illustrating how Lambda containers function or Lambda functions not reacting to events or properly handling CRUD actions (55%)	Does not attempt criterion (0%)	20
Serverless API	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Migrates all of the server-side business logic; applies principles of serverless API and demonstrates how they give the correct results when Lambda executes (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include one of these three examples when code executes on Lambda: call the API, the connection from Lambda to DB; Gateway to Lambda; or User to Gateway (55%)	Does not attempt criterion (0%)	15

Criteria	Exceeds Expectations	Meets Expectations	Partially Meets Expectations	Does Not Meet Expectations	Value
Serverless Database	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Correctly creates a database with the appropriate structure that is able to perform the CRUD functions (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include an incorrect structure or the inability to perform any of the CRUD functions (55%)	Does not attempt criterion (0%)	20
CORS	N/A	Manually enables CORS as defined on the CORS on AWS API Gateway website (100%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include incorrect or partially complete CORS configuration (55%)	Does not attempt criterion (0%)	10
Create and Deploy API	N/A	Deploys the completed API using the "api" deployment name (100%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include incorrect, partially complete, or not deployed APIs (55%)	Does not attempt criterion (0%)	10
Modify and Test Frontend	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Applies changes to the LoopBack SDK configuration in the client, modifies environment products, and builds and deploys the Angular app; the Angular application runs on AWS, calling through Lambda to DynamoDB, and successfully allows CRUD functions on Question and Answer (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include not changing the configuration of the URL to the AWS APIs, not changing the LoopBack configuration to use query parameters instead of headers, failing to build the Angular application before copying to AWS S3, or failing to copy the right files to AWS S3 (55%)	Does not attempt criterion (0%)	10

Criteria	Exceeds Expectations	Meets Expectations	Partially Meets Expectations	Does Not Meet Expectations	Value
Cloud Security	N/A	Sets the permissions through policies and configures them to enable all of the other services with access to read-write and read-only (100%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include creating most or all of the roles and policies, but not configuring all of them correctly (55%)	Does not attempt criterion (0%)	10
Total:					100%